To resolve this, Jane extended the global `Window` interface using the `declare global` block. This technique, known as declaration merging, allows TypeScript to recognize custom properties on the global `window` object, ensuring type safety and preventing errors. By declaring these properties, Jane enabled TypeScript to understand that `window.YT` and `window.onYouTubeIframeAPIReady` exist and have specific types, which eliminated the type errors and improved code reliability. Listing 3.21 shows Jane's type definition.

Listing 3.21 Declare the global YT namespace for window.YT and window.onYouTubeIframeAPIReady

```
declare global { #A
  interface Window { #B
    YT: { #C
      Player: new ( #D
        element: HTMLElement | null,
        config: {
          height: string; #E
          width: string; #F
          videoId: string; #G
          events: {
            onReady: (event: { target: { playVideo: () => void }
          };
          playerVars: { #I
            autoplay: number; #J
            mute: number; #K
          };
        }
      ) => void;
    };
    onYouTubeIframeAPIReady?: () => void;   #L
  }
}
```

Jane clarified that `Window` is a TypeScript interface that defines the structure of the global `window` object provided by the browser. By extending this interface, she informed TypeScript about new properties such as `YT` and `onYouTubeIframeAPIReady`. While `window` is the runtime global variable representing the actual browser environment, TypeScript uses the `Window` interface for static type checking during development—without affecting runtime behavior.